

Системне програмне забезпечення (Операційні системи)

Лабораторна робота №4

Файлова система (частина 1)

У даній лабораторній роботі необхідно розробити драйвер файлової системи для носія інформації, де вміст зберігається в блоках, та частину підсистеми ядра, яка підтримує файлові системи (не зовсім реальний драйвер та частину ядра, але програма, яку треба розробити, за функціональністю має бути дуже наближеною до цих програм). Введення-виведення даних з носієм інформації, де вміст зберігається в блоках, відбувається блоками, усі блоки мають однакові розміри, розмір блоку визначається апаратурою. Зазвичай у таких пристроях не враховуються обмеження на кількість змін в одному блоці.

У ФС є два типи файлів: *звичайний файл* (regular) та *директорія* (directory). У цій лабораторній роботі необхідно реалізувати ФС з однією директорією, яка «містить» звичайні файли.

Вміст кожного файлу зберігається в блоках, один блок містить інформацію лише одного файлу. Для обліку зайнятих блоків використовується бітова карта, значення одного біту визначає чи є відповідний блок зайнятий. Оскільки кількість блоків у носії інформації не змінюється, тому бітова карта буде містити постійну кількість бітів. Останній блок файлу може використовуватися частково. Блоки в структурах даних ФС адресуються за їхніми порядковими номерами.

Кожен файл, як об'єкт ФС, представлений дескриптором файлу. Кількість дескрипторів файлів вказується заздалегідь, тому не може бути створено більше певної кількості файлів у ФС, навіть якщо в носії інформації є вільне місце. Усі дескриптори файлів зберігаються у масиві, що дозволяє визначити позицію дескриптора файлу на носії інформації за його номером. Дескриптор файлу повинен містити принаймні таку інформацію: тип файлу (звичайний файл або директорія), кількість жорстких посилань на файл, розмір файлу в байтах, мапу номерів блоків файлу. Карта номерів блоків має наступну структуру: є кілька прямих посилань на блоки та є одне посилання на блок, який містить прямі посилання на блоки (посилання на блок – це номер блоку). Індекс посилання в мапі визначає зміщення даних у файлі. Посилання не вказує на блок у двох випадках: відповідне зміщення перевищує розмір файлу або значення всіх байтів у відповідному блоці дорівнюють нулю.

Звичайний файл – це тип файлу, який використовуються для зберігання будь-яких даних. Драйвер ФС не інтерпретує його вміст, ядро не інтерпретує вміст довільного звичайного файлу.

Директорія – це тип файлу, вміст якого складається із записів директорії (directory entry). Формат запису директорії визначається форматом ФС та інтерпретується лише драйвером ФС. Запис директорії (directory entry) називається *жорстке посилання* (hard link). Запис директорії складається з імені файлу та номера дескриптора файлу. Максимальна довжина імені файлу та дозволені символи в ньому визначаються форматом ФС. Оскільки ім'я файлу не є частиною дескриптора файлу, тому кілька жорстких посилань можуть вказувати на один файл, тобто один файл може мати кілька імен. Оскільки жорсткі посилання на файли можуть бути знищені, необхідно передбачити можливість позначити недійсні жорсткі посилання в директорії.

У цій лабораторній роботі не потрібно реалізовувати буферний кеш та кеш імен файлів. Драйвер ФС повинен працювати безпосередньо з даними на носії інформації під час виконання будь-якого запиту.

Завдання на роботу

Розробити драйвер ФС та частину підсистеми ядра, яка підтримує файлові системи, що відповідає наведеній вище семантиці ФС. Драйвер ФС та частину ядра реалізувати в програмі користувача. Використовуйте наведену вище ідею структури дескриптора файлу. Використайте вміст звичайного файлу в наявній ФС в якості вмісту носія інформації, розмір файлу визначає розмір носія інформації. Самостійно вибрати розмір блоку, розмір мапи номерів блоків у дескрипторі файлу, максимальну довжину імені файлу. Розроблена програма не має бути ФС типу `bypass` чи `pass through`, усі дії з ФС мають відбуватися в розробленій програмі. Використовувати FUSE або схожу систему не дозволяється, оскільки в лабораторній роботі треба реалізувати частину підсистеми ядра. Якщо використовувати FUSE, тоді буде використовуватися наявна підсистема ядра.

Замість використання вмісту звичайного файлу в наявній ФС в якості вмісту носія інформації, можна використати пам'ять як носій інформації. Тобто можна розробити драйвер ФС для пам'яті. У цьому випадку бітова карта не потрібна. Для представлення вмісту директорії можна використати щось більш оптимальніше ніж дані в блоках. Замість мапи номерів блоків можна використати будь-яку структуру даних, яка дозволить ефективно кодувати номери блоків та блоки, вміст яких складається з нулів. Кількість дескрипторів файлів та кількість блоків визначати не треба. Вміст звичайного файлу все одно повинен складатися з блоків.

Розробити програму, яка підтримує наступні команди (замість введення команд можна викликати необхідні функції безпосередньо в програмі):

- `mkfs n` – ініціалізувати ФС, `n` – це кількість дескрипторів файлів (ця команда не потрібна для ФС для пам'яті).
- `stat name` – вивести інформацію про файл (дані дескриптору файлу).
- `ls` – вивести список жорстких посилань на файли з номерами дескрипторів файлів в директорії.
- `create name` – створити звичайний файл та створити на нього жорстке посилання з ім'ям `name` у директорії.
- `fd = open name` – відкрити звичайний файл, на який вказує жорстке посилання з ім'ям `name`. Команда повинна призначити найменше вільне невід'ємне цілочисельне значення (назвемо його *числовий дескриптор файлу*) для роботи з відкритим файлом (це число – це не те саме, що номер дескриптору файлу у ФС). Один файл може бути відкритий кілька разів. Кількість числових дескрипторів файлів може бути обмежена.
- `close fd` – закрити раніше відкритий файл з числовим дескриптором файлу `fd`, значення `fd` стає вільним.
- `seek fd offset` – вказати зміщення для відкритого файлу, де почнеться наступне читання або запис (далі «зміщення»). При відкритті файлу зміщення дорівнює нулю. Це зміщення вказується тільки для цього `fd`.
- `read fd size` – прочитати `size` байт даних з відкритого файлу, до значення зміщення додається `size`.
- `write fd size` – записати `size` байт даних у відкритий файл, до значення зміщення додається `size`.
- `link name1 name2` – створити жорстке посилання з ім'ям `name2` на файл, на який вказує жорстке посилання з ім'ям `name1`.
- `unlink name` – знищити жорстке посилання з ім'ям `name`.

- `truncate name size` – змінити розмір файлу, на який вказує жорстке посилання з ім'ям `name`. Якщо розмір файлу збільшується, тоді неініціалізовані дані дорівнюють нулям.

Драйвер ФС звільнює дані файлу, якщо жодне жорстке посилання не вказує на файл і файл не є відкритим. На файл може не вказувати жодне жорстке посилання, але якщо він відкритий, тоді з ним можна виконувати команди `seek`, `read` та `write`.

Якщо команда `truncate` збільшує розмір файлу, тоді реалізувати оптимізацію та не створювати блоки, вміст яких складається з нулів.

Звіт

- Опис форматів даних ФС (за бажанням).
- Лістинг програми.
- Приклади виконання всіх команд.

Література

- Modern Operating Systems. 4th edition. Tanenbaum A.